Aufgabe 1: Drehfreudig

Team-ID: 00562

Team-Name: Vikkrck07

Bearbeiter/-innen dieser Aufgabe:
Kristof Leidig

16. November 2025

Inhaltsverzeichnis

1 Lösungsidee **1**
1.1 Das Erkennen . . . 1
1.1.1 Die gewählte Lösung . . . 1
1.1.2 Die Alternative . . . 2
1.2 Die Visualisierung . . . 2

2 Umsetzung **2**

3 Werkzeuge **3**

4 Beispiele **3**

5 Quellcode **6**

1 Lösungsidee

Wie auch die Aufgabe selbst wird die Lösung von mir in zweierlei Teile aufgeteilt.

1.1 Das Erkennen

Hier stellte sich die Frage: Wie erkenne ich, ob ein Baum drehfreudig ist?

1.1.1 Die gewählte Lösung

Mein Ansatz war es, den Baum nicht wirklich als Datenstruktur zu bauen, sondern die Prüfung direkt auf dem String durchzuführen. Auch wenn dies mir möglicherweise den Ansatz etwas schwerer umzusetzen machte, hielt ich es für eine interessante Herausforderung. Grundsätzlich muss ich also trotzdem über den String iterieren, speichere dabei aber nur die aktuell nötigen Daten und nicht die gesamte Struktur. Zur Erkennung muss ich also folgende drei Schritte durchführen:

1. Eine ursprünglich aufgerufene Methode, die eine Rekursion anstößt. Mit einer von der Rekursion erhaltenen Liste an Blättern wird deren Breite und Höhe auf Palindromeigenschaften geprüft. Laufzeit: $O(n + r)$ [n als Länge des Strings, r als Zeitkomplexität der Rekursion], da die Anzahl an Blättern hier als proportional zur Länge des Strings betrachtet wird und die erhaltene Liste zugleich von hinten und vorne traversiert wird. Hier wird die Annahme getroffen, dass es sich um eine Abwandlung eines Arrays handelt und somit der Zugriff auf i in O(1) erfolgt.

2. Eine rekursive Methode, aufgerufen vom ersten Schritt, muss nun den Baum traversieren. Diese Methode muss nun die Breite und Tiefe des aktuellen Knotens berücksichtigen und beim Erreichen von Blättern diese Informationen in eine Liste schreiben. Rekursionsabbruch ist das vollständige Traversieren der Liste, und es muss eine Methode aufgerufen werden, die die Breite der Kinder bestimmen kann. Laufzeit: $O(n \cdot k)$ [n als Länge des Strings, k als Zeitkomplexität der aufgerufenen Methode]

3. Letztlich noch die Methode zum Finden der direkten Kinder, die recht simpel die öffnenden Klammern auf der jeweils nächsten Ëbeneßählt. Laufzeit: $O(n)$ [n als Länge des Strings]

Nach Vereinigung der zuvor genannten Methoden ergibt sich ein Wahrheitswert, der die "Drehfreudigkeit"beschreibt. Die Laufzeit sollte vereinigt $O(n^2)$ betragen, da die rekursive Methode dominiert.

1.1.2 Die Alternative

Wie zuvor erwähnt, könnte man auch eine Baumstruktur erstellen. Dies wurde aufgrund eigener Präferenz jedoch recht schnell unterlassen, da ich es langweilig fand. Die Funktionsweise des Erkennens sollte sich jedoch nicht maßgeblich verändern.

1.2 Die Visualisierung

Für die Visualisierung habe ich mich für Java mit Swing entschieden, da ich hiermit bereits Erfahrung habe. Die Visualisierung erfolgt ebenfalls ohne explizite Baumstruktur direkt auf Basis des Strings.

Zunächst werden alle Klammerpaare mit ihren Positionen im String gesammelt und sortiert. Dies ergibt eine Liste von Knoten mit ihren Start- und End-Indizes. Eine weitere Funktion bestimmt für jeden Knoten seine direkten Kinder, indem sie die Knotenliste durchsucht.

Die eigentliche Zeichnung erfolgt rekursiv, wobei jeder Knoten sich selbst als Rechteck (bei Blättern verlängert bis zur maximalen Tiefe) mit einem Kreis in der Mitte zeichnet, sich mit seinem Elternknoten verbindet und sich dann rekursiv für alle Kinder aufruft. Die Breite wird dabei gleichmäßig auf die Kinder aufgeteilt, wie in der Aufgabe beschrieben, und da ich mir nicht bewusst bin, wie ich eine bestehende G2D-Grafik rotiere, muss das Zeichnen zweifach passieren.

Die Laufzeit der Visualisierung wird bestimmt durch:

1. Sammeln der gesamten Knoten und Sortieren dieser. Hierfür habe ich letztlich aufgrund der Unmöglichkeit einer verkehrt sortierten Liste und der kleinen Datenmengen Insertion-Sort gewählt. Hier dominiert also das Sortieren. Laufzeit: $O(n^2)$ [n als Länge des Strings (da Anzahl Knoten = Stringlänge/2)]

2. Mit dieser Liste wird dann eine rekursive Methode aufgerufen, die diese Liste traversiert und jeden Knoten in Reihenfolge der Liste malt. Für jeden rekursiven Aufruf muss aber herausgefunden werden, wie viele Kinder der aktuelle Knoten hat. Laufzeit: $O(n)$ [mit n als Länge des Strings (vgl. voriger Schritt)]

3. Letztlich noch die Methode zum Finden der Kinder. Diese ist überraschend einfach, da die Liste der Knoten sortiert ist und mit bekanntem Index des aktuellen Knotens von dessen Anfang bis zum Ende gesucht werden muss. Wichtig ist auch, Enkelknoten können durch geschicktes Anpassen des Zählers übersprungen werden, was heißt, man muss für den gesamten rekursiven Durchlauf jeden Knoten nur einmal besuchen mit dieser Methode. Laufzeit: $O(n)$ für alle Aufrufe zusammen [mit n als Länge des Strings (vgl. voriger Schritt)]

Die Gesamtlaufzeit der Visualisierung beträgt somit $O(n^2)$, dominiert von Insertion-Sort. Warum also nicht Quick- oder Merge-Sort? Aufgrund der Datenmenge an fast sortierten Knoten bzw. der Aneinanderreihung von sortierten Teilsegmenten kann im Folgenden angenommen werden, dass Insertion-Sort deutlich näher an $O(n)$ läuft, was eine entsprechende Gesamtlaufzeit in der gleichen Größenordnung zur Folge hat. (bzwp. dem Doppelten, da es zweifach aufgerufen wird)

2 Umsetzung

Die Umsetzung wurde zu großen Teilen bereits erläutert, aber ein paar Kommentare dazu gibt es dennoch. Das erklärte Konzept war im Großen und Ganzen von Anfang an klar, also ergaben sich keine fundamentalen Änderungen in der Entwicklung, nachdem einmal angefangen wurde, die Erkennung in Java

zu schreiben. Es wurde oft erst die Funktionalität implementiert und folglich JUnit-Tests geschrieben, was normalerweise die falsche Herangehensweise ist. Jedoch konnten bei dieser Aufgabe die Tests ohnehin nicht der Implementierung angepasst werden, da das Ziel von Anfang an feststand. Somit habe ich vermieden, Tests mit falschen Datentypen zu schreiben, da die Datentypen bereits feststanden. Letztlich erwähnenswert bei der Umsetzung ist, dass sie auf meinem GitHub dokumentiert ist über sämtliche Push-Nachrichten und dass die Erkennung zeitlich mit recht großem Abstand zur Visualisierung implementiert wurde, da ich mir zuvor mit Swing etwas vertrauter machen musste.

3 Werkzeuge

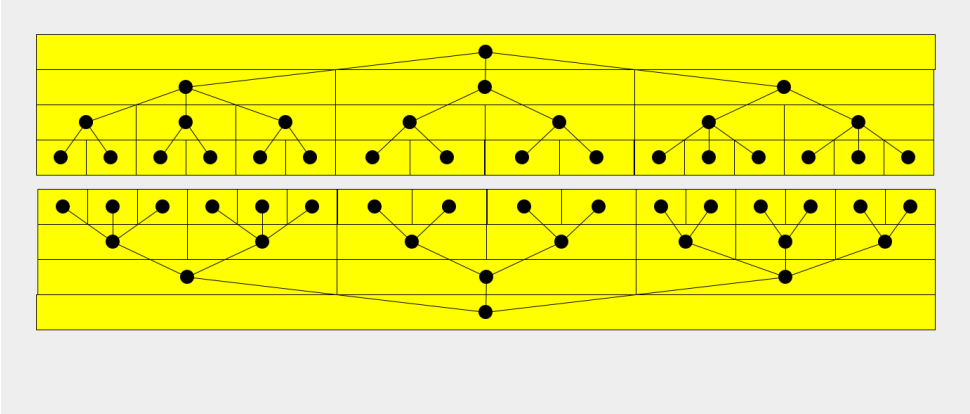
Das Konzept wurde von mir selbst entwickelt und damit ohne Hilfe von jeglicher KI. Zumeist heißt das, ich habe Zeichnungen in MS Paint angefertigt und versucht zu verstehen, wie man diese Aufgabe algorithmisch lösen kann. Oft habe ich dabei die Probleme mir selbst, einem der Teamkollegen oder meinem Vater erklärt, weil mir dies etwas mehr Verständnis gab. Teile habe ich handschriftlich auf Papier verewigt, und diese Seiten werden bei der Quellcode-Sektion als Bilder angehängt. Bei der Erkennung habe ich mich dabei hauptsächlich darauf fokussiert, ob die Tiefe eines Baumes wichtig ist und wie man am besten den String traversiert. Bei der Programmierung wurde auch nur spärlich KI verwendet, namentlich Claude, um bestimmte Fehler oder Exceptions zu identifizieren, auf die ich traf. Letztlich kann ich also garantieren, dass der Code gänzlich von mir geschrieben ist. Was Librarys angeht, muss ich erwähnen, dass ich wie beschrieben Swing benutzt habe, um die Visualisierung zu ermöglichen; sonst wurden keine nennenswerten Bibliotheken benutzt. Für diese Dokumentation wurde aber wieder auf KI zurückgegriffen, da meine Leistungen im Fach Deutsch nicht gerade ausreichend sind, um einen Text mit adäquater Rechtschreibung zu gestalten, weshalb ich sagen muss, dass jeder Paragraph nach dem Schreiben auf diese geprüft wurde.

4 Beispiele

Für diesen Paragraphen kann ich gerne auf die Tests hinweisen, die fast alle Komponenten testen, aber im folgenden wird auch kurz auf die Beispiele eingegangen.

1. "drehfreudig01.txt Das einfache Beispiel

   Hier ist nicht viel zu sagen, da dieses Beispiel bereits in den Aufgaben gezeigt wird, aber es generiert den gezeigten Baum und beweist damit, dass er drehfreudig ist.



2. "drehfreudig02.txt Das einfachste falsche Beispiel

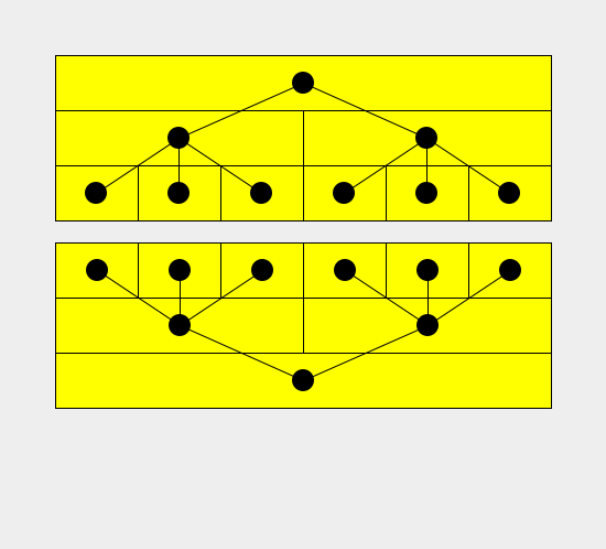
   Hierzu ist auch wenig zu sagen, da es ebenfalls gestellt wird und einfach unterschiedliche Breiten der Rechtecke hat. Weil der Baum keine Drehfreudigkeit zeigt wird auch kein Bild generiert und in der Konsole "falseäusgegeben.

3. "drehfreudig03.txt Ein weiteres Negativbeispiel

   Dieses wurde (Überraschung) ebenfalls gestellt und zeigt einen Baum bei dem in der Mitte weder Breite noch Höhe passt, demnach also keine Fehlerquelle bei jeglichen Implimentierungen.

4. "drehfreudig04.txt Ein drehfreudiger Baum :3

   Das erste neue Beispiel, was ebenfalls keine Probleme machte aber recht schön ausieht und regelmäßig ist.



5. "drehfreudig05.txt Ein weiteres Negativbeispiel

   Dieses wurde nicht gestellt aber zeigt auch leider nichts neues, also einfache "false".

6. "drehfreudig06.txt Ein interessantes Negativbeispiel

   Endlich ist nicht die Breite eines Rechteckes entscheidend für die Ausgabe sondern die Höhe, denn die führt bei diesem Beispiel zu der Ausgabe "falseïn der Konsole. (3 != 4)

7. "drehfreudig07.txt Ein ehr langweiliges Negativbeispiel

   Hier passt weder die Breite noch die Höhe der Rechtecke, was heißt nur eines der Beiden hätte implementiert sein müssen, was diese Aufgabe recht langweilig machte, von technischer Seite jedenfalls.

8. "drehfreudig08.txt Ein weiteres Negativbeispiel

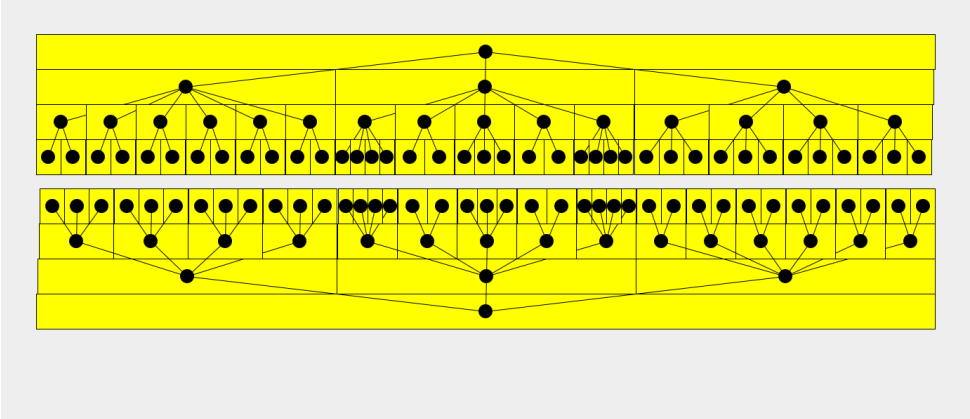
   Dieses Beispiel gibt Flashbacks von Beispiel 6, da es einfach nur ein größerer Baum mit der selben Problematik ist.

9. "drehfreudig09.txt Ein knappes Negativbeispiel

   Dieses Beispiel sieht fast so aus wie das erste, jedoch die Breite in dem Mittleren Teil stimmt nicht überein.

10. "drehfreudig10.txt Ein drehfreudiger Baum :3

    Der letzte Baum der gegebenen Beispiele der die Eigenschaft erfüllt. Sieht er nicht wunderschön aus? Er ist das nächste, was annähernd an die Laufzeitsgrenze gelangt, da er der größte Baum ist, der visualisiert werden muss.



11. "drehfreudig011.txt Ein einfaches Negativbeispiel

    Hier entspricht der linke Teilbaum keineswegs dem Rechten, was zu einem leichten Ausschluss des drehfreudig Kriteriums führt.

12. "drehfreudig012.txt Ein knappes Negativbeispiel

    Eine kleine Inkonsistenz im rechten Teilbaum führt dazu, dass dieser Baum die Anforderungen nicht erfüllt.

13. "drehfreudig013.txt Ein einfaches Negativbeispiel

    Hier stimmen ein weiteres Mal die Höhen der Teilbäume nicht überein

14. "drehfreudig014.txt Ein einfaches Negativbeispiel

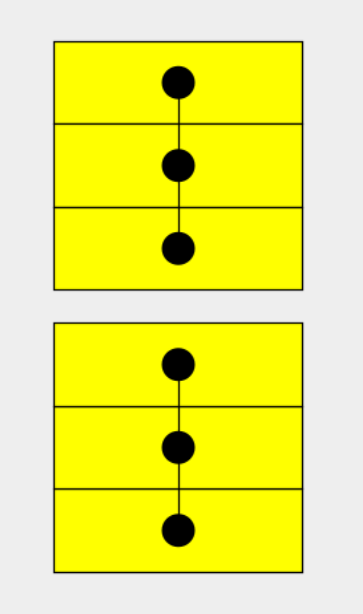
    Hier passen erneut die breiten nicht, also erneut keine Herausforderung.

15. "drehfreudig015.txt Ein einfaches Negativbeispiel

    Erneut eine sehr einfacher Baum, da die Höhen einfach nicht übereinstimmen

16. Bonus: ((())) - Ein entarteter Baum

    Dieser Baum ist entartet und somit drehfreudig. Generell kein schwer zu testendes Beispiel aber trotzdem ganz interessant. Auch dieses Beispiel wird einfach erkannt.



5 Quellcode

```
import Helper.Pair;
import Helper.Panel;
import Helper.Rational;

import java.util.ArrayList;
import java.util.List;
import java.util.Objects;

public class Brain {
    public static Pair<Boolean,Integer> isDrehfreudig(String tree){
        if (tree == null) return new Pair<>(false,-1);
        if (tree.isEmpty()) return new Pair<>(false,-1);
        if (tree.equals("()")) return new Pair<>(true,1);

        List<Pair<Rational,Integer>> info = calcInfo(tree);

        int i = info.size();
        for (int j = 0; j < i/2; j++) {
            if (!info.get(j).a().equals(info.get(i-1-j).a()) || !Objects.equals(info.get(j).b(),
            info.get(i - 1 - j).b())){
                return new Pair<>(false,-1);
            }
        }

        int depth = 1;
        for (Pair<Rational,Integer> p:info) if (p.b()> depth) depth = p.b();

        return new Pair<>(true, depth);
    }

    private static List<Pair<Rational,Integer>> calcInfo(String tree){
        List<Pair<Rational,Integer>> info = new ArrayList<>();
        calcInfoRec(tree, 0, tree.length()-1, new Rational(1,1),1, info);
        return info;
    }

    private static void calcInfoRec(String tree, int start, int end, Rational width, int depth,
    List<Pair<Rational,Integer>> info){
        if (start >= end) return;

        if (end-start == 1 && tree.charAt(start) == '(' && tree.charAt(end) == ')'){
            info.add(new Pair<>(width,depth));
            return;
        }

        List<Pair<Integer,Integer>> children = findChildren(tree, start, end);

        if (children.isEmpty()) return;

        Rational childWidth = width.divide(children.size());

        for (Pair<Integer,Integer> p : children) {
            calcInfoRec(tree, p.a(), p.b(), childWidth, depth+1, info);
        }
    }

    private static List<Pair<Integer,Integer>> findChildren(String tree, int start, int end){
        List<Pair<Integer,Integer>> range = new ArrayList<>();

        int pos = start+1;
        while (pos < end) {
            if (tree.charAt(pos) == '('){
                int child = pos;
                int depth = 1;
                pos++;

                while (pos < end && depth > 0) {
                    if (tree.charAt(pos) == '(') depth++;
                    else depth--;
                    pos++;
                }
```

```
                range.add(new Pair<>(child, pos - 1));
            }
            else {
                pos++;
            }
        }
        return range;
    }

    public static void main(String[] args) {
        String tree = Reader.readString(args[0]);
        Pair<Boolean,Integer> res = isDrehfreudig(tree);
        if (res.a()){
        try{ImageExport.exportToFile(new Panel(res.b(),tree), "tree_output.png");}
        catch (Exception e){ throw new RuntimeException();}}
        else System.out.println(false);
    }
}
```

listing

```
package Helper;

import Exeptions.InvalidStringException;

import javax.swing.*;
import java.awt.*;
import java.awt.geom.AffineTransform;
import java.util.ArrayList;
import java.util.List;
import java.util.Stack;

public class Panel extends JPanel {

    private final int maxDepth;
    private final String tree;
    private final int maxWidth;

    public Panel(int maxDepth, String tree){
        maxWidth = Math.min((tree.length()/2)*50,1280);
        this.setPreferredSize(new Dimension(maxWidth+100, Math.min(maxDepth*50*2+200,1280)));
        this.maxDepth = maxDepth;
        this.tree = tree;

    }

    public static List<Integer> findChildren(int nodeInd, List<Pair<Integer,Integer>> nodes) {
        List<Integer> c = new ArrayList<>();

        if (nodes.size() <= nodeInd + 1) return c;

        //I am your father again :3
        Pair<Integer,Integer> father = nodes.get(nodeInd);
        int a = father.a();
        int b = father.b();

        int skip = a;

        for (int i = nodeInd + 1; i < nodes.size(); i++) {
            Pair<Integer,Integer> node = nodes.get(i);
            int open = node.a();
            int close = node.b();

            if (open >= b) break;

            if (open < skip) continue;

            c.add(i);

            skip = close;
        }
```

```
52          return c;
        }
54
        private void rec(Graphics2D g, int x, int y, Rational width,
56                       Pair<Integer,Integer> parentCenter,
                         List<Pair<Integer,Integer>> nodes, int nodeIdx,
58                       int currentDepth, int maxDepth) {

60          int boxHeight = 50;

62          List<Integer> children = findChildren(nodeIdx, nodes);
            boolean isLeaf = children.isEmpty();
64
            int rectHeight;
66          if (isLeaf) {
                int remainingLevels = maxDepth - currentDepth;
68              rectHeight = boxHeight + (remainingLevels * boxHeight);
            } else {
70              rectHeight = boxHeight;
            }
72
            Rectangle r = new Rectangle(x, y, width.multiply(maxWidth), rectHeight);
74          g.setColor(Color.yellow);
            g.fill(r);
76          g.setColor(Color.black);
            g.draw(r);
78
            int centerX = x + width.multiply(maxWidth) / 2;
80          int centerY = y + boxHeight / 2;
            g.fillOval(centerX - 10, centerY - 10, 20, 20);
82
            if (parentCenter != null) {
84              g.drawLine(parentCenter.a(), parentCenter.b(), centerX, centerY);
            }
86
            if (!isLeaf) {
88              Rational childWidth = width.divide(children.size());
                int childY = y + boxHeight;
90
                for (int i = 0; i < children.size(); i++) {
92                  int childX = x + childWidth.multiply(maxWidth)*i;
                    rec(g, childX, childY, childWidth,
94                          new Pair<>(centerX, centerY),
                            nodes, children.get(i),
96                          currentDepth + 1, maxDepth);
                }
98          }
        }
100
        public static List<Pair<Integer,Integer>> collectNodes(String s){
102         List<Pair<Integer,Integer>> n = new ArrayList<>();
            int c = 0;
104         Stack<Pair<Integer,Integer>> stack = new Stack<>();

106         for (int i = 0; i < s.length(); i++) {
                if (s.charAt(i) == '(') stack.push(new Pair<>(c++, i));
108             else if (s.charAt(i) == ')') {
                    Pair<Integer,Integer> st = stack.pop();
110                 n.add(new Pair<>(st.b(), i));
                    c--;
112             }
                else throw new InvalidStringException("The string is invalid");
114         }

116         InsSort.sort(n);

118         return n;
        }
120
        @Override
122     public void paint(Graphics gx) {
            super.paint(gx);
```

```
124         Graphics2D g = (Graphics2D) gx;
            g.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
126
            List<Pair<Integer,Integer>> nodes = collectNodes(tree);
128
            if (!nodes.isEmpty()) {
130             int rootInd = 0;
                int treeHeight = maxDepth*50*2;
132             int gap = 20;

134             rec(g, 50, 50, new Rational(1,1), null, nodes, rootInd, 1, maxDepth);

136             AffineTransform originalTransform = g.getTransform();

138             int rotY = 50 + treeHeight + gap;
                int rotX = getWidth() / 2;
140
                g.translate(rotX, rotY);
142             g.rotate(Math.PI);
                g.translate(-rotX, -50);
144
                rec(g, 50, 50, new Rational(1,1), null, nodes, rootInd, 1, maxDepth);
146
                g.setTransform(originalTransform);
148         }
        }
150 }
```

PS. Zum testen mit neuen Dateien das .jar file über gradle neu bauen bitte. Die benutzte Java Version ist Java 19.